AI TRANSIT PIPELINE

Securing AI Code Integration in the Enterprise

- Secure retrieval from GitHub
- 5-layer adaptive multi-layer scan by file type
- Approved ZIP archive + Excel traceability report

Table of Contents

01 Context & Objectives

02 Pipeline Architecture — 5 Layers

03 Phase 1 — Secure Retrieval

04 Layer 1 — Global Scan

05 Layer 2 — OWASP Top 10 / CWE Top 25

06 Layer 3 — SCA: Vulnerable Dependencies

07 Layer 4 — Universal Patterns

08 Layer 5 — Per-type SAST (11 languages)

09 Tools Used

10 Results & Deliverables

11 Absolute Security Rules

01 Context & Objectives

Why this pipeline?

- AI-generated code may contain backdoors, secrets, or dangerous patterns
- Developers import code without systematic security review
- The corporate network must remain isolated (partial air-gap)
- No traceability without a structured process



Key Objectives

Retrieve

Secure depth-1 clone

Scan

5-layer adaptive pipeline

Decide

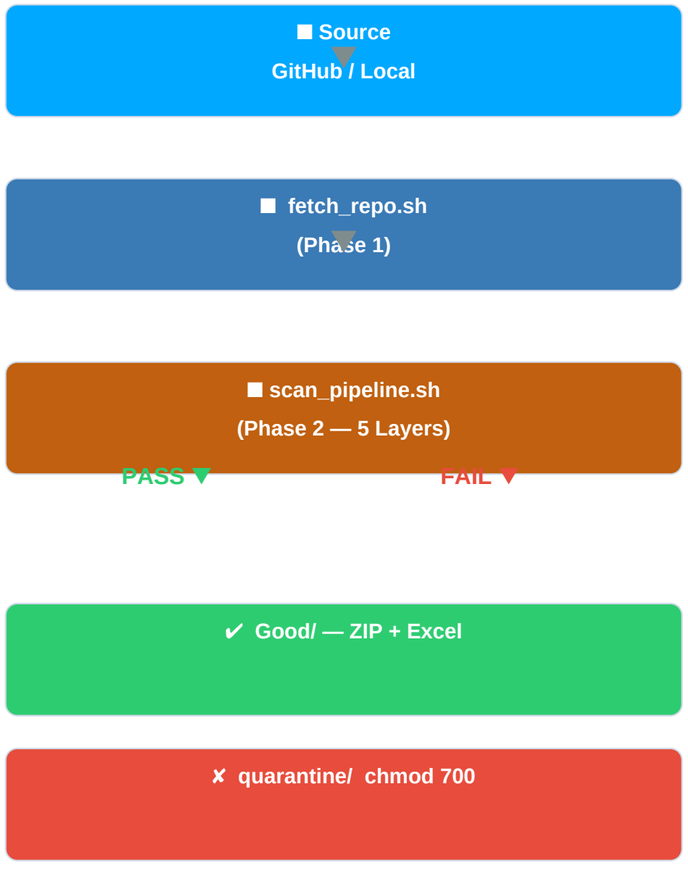
PASS→ZIP / FAIL→Quarantine

Trace

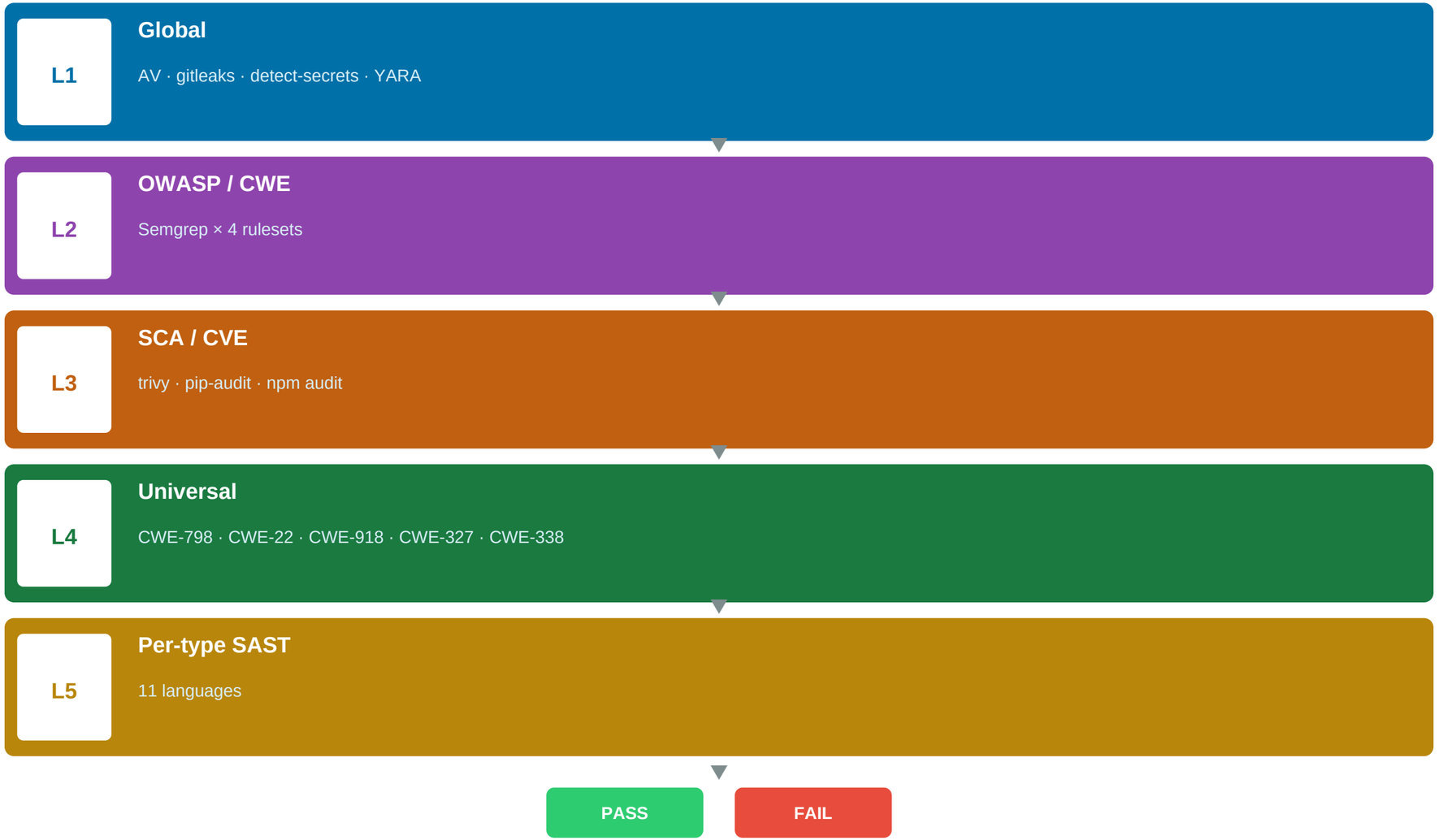
Excel + JSON + HTML

02 Pipeline Architecture — 5-Layer Scanning

Top-level Flow



5-Layer Scanning Cascade



03 Phase 1 — Secure Retrieval

1 Host Whitelist

Only github.com is allowed.

Any other URL is rejected immediately.

2 Size Verification

GitHub API queried before clone.

Limit: 500 MB.

3 Minimal Clone

git clone --depth 1 --no-tags

--single-branch

4 .git/ Removal

Git metadata removed

(hooks, remotes, submodules).

5 SHA-256 Manifest

Hash of each file →

.manifest_sha256.txt (audit trail).

6 Quick Triage

Grep: eval(exec(curl|bash

rm -rf → immediate alert.

04 Layer 1 — Global Scan (all files)

AV · gitleaks · detect-secrets · YARA — Applies to the ENTIRE directory, regardless of file type

■ Gitleaks

- > 150 credential types
- GitHub/AWS/GCP/Azure tokens
- RSA, PEM, SSH, JWT keys
- Entropy analysis + regex

FAIL if positive

■ detect-secrets

- Shannon entropy
- Secrets unknown to rule sets
- High-density base64/hex
- Complements Gitleaks

FAIL if positive

■ ClamAV / AV

- Millions of signatures
- Trojans, ransomware, backdoors
- Full recursive scan
- freshclam updated base

FAIL if positive

■ YARA

- Industry-standard IOC
- Custom .yar rules
- AI backdoor patterns
- Regex + boolean logic

FAIL if positive

05 Layer 2 — OWASP Top 10 2021 + CWE Top 25 (Semgrep)

4 Semgrep rulesets run against all code files

p/owasp-top-ten

OWASP Top 10 2021

- A01 — Broken Access Control
- A02 — Cryptographic Failures
- A03 — Injection (SQLi, XSS, SSTI…)
- A04 — Insecure Design
- A05 — Security Misconfiguration
- A06 — Vulnerable Components
- A07 — Auth Failures
- A08 — Data Integrity Failures
- A09 — Logging & Monitoring Failures
- A10 — SSRF

p/cwe-top-25

CWE Top 25 Most Dangerous

- CWE-79 XSS
- CWE-89 SQL Injection
- CWE-20 Improper Input Validation
- CWE-125 Out-of-bounds Read
- CWE-78 OS Command Injection
- CWE-416 Use After Free
- CWE-22 Path Traversal
- CWE-352 CSRF
- CWE-476 NULL Pointer Dereference
- CWE-287 Improper Authentication

p/security-audit

General Security Audit

- Broad vulnerability patterns
- Misconfigurations
- Insecure API usage
- Deprecated functions
- Crypto weaknesses
- Race conditions
- Integer overflows
- Memory safety issues
- Protocol misuse
- Hardcoded values

p/secrets

Secrets & Credentials

- API keys (AWS, GCP, Azure…)
- Private keys (RSA, EC, PEM)
- OAuth tokens
- Database connection strings
- JWT secrets
- Slack / Twilio / Stripe tokens
- Generic password patterns
- High-entropy strings
- Environment variable leaks
- Config file secrets

06 Layer 3 — SCA: Vulnerable Dependencies (OWASP A06)

What is SCA?

Software Composition Analysis (SCA) identifies known vulnerabilities (CVEs) in third-party libraries and dependencies.

OWASP A06:2021 — Vulnerable and Outdated Components is consistently in the OWASP Top 10.

■ trivy

Universal SCA scanner

- All ecosystems (pip, npm, gem, go, cargo…)
- CVE database (NVD, OSV, GitHub Advisory)
- Scans requirements.txt, package-lock.json,
- go.sum, Gemfile.lock, Cargo.lock…
- CRITICAL / HIGH / MEDIUM / LOW severity
- Output: JSON + table report
- Also scans Dockerfiles & container images
- Regularly updated vulnerability DB

FAIL on CRITICAL/HIGH

■ pip-audit / safety

Python — requirements*.txt

- pip-audit: official PyPA tool (PURL-based)
- safety: PyUp.io database
- Scans requirements.txt, requirements-dev.txt
- requirements-*.txt (glob pattern)
- Maps to CVE + GHSA identifiers
- Detects yanked / abandoned packages
- Outputs FAIL on any CRITICAL finding
- Complements trivy for Python ecosystem

FAIL on CRITICAL/HIGH

■ npm audit

JavaScript / Node — package-lock.json

- Built-in npm command (no install needed)
- Uses npm registry advisory database
- Scans package-lock.json
- Severity: critical / high / moderate / low
- FAIL on critical or high findings
- Reports affected package tree
- Identifies fix version when available
- Covers direct and transitive dependencies

FAIL on CRITICAL/HIGH

07 Layer 4 — Universal Patterns (all files)

Pattern-matched against EVERY file regardless of language — no tool dependency

CWE / Standard	Pattern / Rule	Example	Verdict
CWE-798	Hardcoded credentials	password="...", api_key="..."	FAIL
CWE-321	Hardcoded crypto key	BEGIN RSA PRIVATE KEY	FAIL
CWE-22	Path traversal	../../etc/passwd	FAIL
CWE-918	SSRF	requests.get(url=user_input)	WARN
CWE-327	Weak cryptography	md5, sha1, rc4	WARN
CWE-338	Weak PRNG	Math.random(), rand()	WARN
OWASP-A09	Logging suppressed	logging.disable(logging.CRITICAL)	WARN

08 Layer 5 — Per-type SAST Dispatch (11 languages)

Extension(s)	SAST Tool	Manual Checks	Verdict
.py	Bandit	eval() exec() import os/subprocess	FAIL
.js .ts .jsx	Semgrep	eval() child_process require(…)	FAIL
.java	Semgrep	Runtime.exec() ObjectInputStream JNDI Log4Shell TrustAllCerts	FAIL
.php	Semgrep	shell_exec() echo $_GET include($var) unserialize()	FAIL
.rb	Semgrep	system/exec eval() ActiveRecord string interpolation	FAIL
.go	Semgrep	fmt.Sprintf in SQL math/rand TLS MinVersion	FAIL
.c .cpp .h	cppcheck	gets() strcpy() system() popen()	FAIL
.sh .bash .ps1	ShellCheck	curl|bash wget|sh eval $var	FAIL
.yml .yaml	—	Unpinned actions inline secrets	FAIL
.tf .tfvars	checkov	Hardcoded secrets overly broad IAM	FAIL
Dockerfile	hadolint	:latest ADD http:// RUN curl|bash	FAIL
.xml	—	DOCTYPE + ENTITY (XXE CWE-611)	FAIL
.so .exe .elf	strings	Auto FAIL + IOC strings	FAIL ■
.zip .tar.gz	—	Auto FAIL — re-scan after extraction	FAIL ■
.sql	—	xp_cmdshell DROP TABLE ; --	FAIL

08 Layer 5 — Per-type Check Details

Python .py

Dynamic execution — high risk of backdoors via eval/exec

Bandit SAST

FAIL

Python AST: subprocess shell=True, pickle, MD5, SQL, assert…

eval() / exec()

FAIL

Arbitrary execution at runtime — trivial backdoor vector

import os/subprocess

WARN

Shell access — requires manual review

JavaScript .js .ts

Ecosystem exposed to supply-chain attacks

Semgrep p/javascript

FAIL

XSS, prototype pollution, innerHTML, SSTI…

eval()

FAIL

Dynamic execution — classic injection vector

child_process

FAIL

Shell command execution from Node.js

08 Layer 5 — Per-type Check Details

C / C++ .c .cpp

Risks from manual memory management

cppcheck

FAIL

Buffer overflows, null ptr, use-after-free, uninit memory.

gets() strcpy()

FAIL

Unbounded read/copy → buffer overflow

system() popen()

FAIL

Shell command execution → possible injection

Shell .sh .bash

Direct execution — command obfuscation is trivial

ShellCheck

FAIL

Unquoted variables, globbing, dangerous redirections…

curl | bash

FAIL

Remote code execution without integrity check

eval $variable

FAIL

Trivial injection if variable controlled by attacker

08 Layer 5 — Per-type Check Details

YAML / CI .yml

CI/CD pipelines — a bad config opens backdoors

Unpinned actions

FAIL

uses: action@main → vulnerable to tag-hijacking

Inline secrets

FAIL

password: value (not ${{ secrets.XXX }})

Terraform .tf

IaC — can provision entire cloud resources

checkov (CIS)

FAIL

S3 buckets, overly broad IAM, DB without at-rest encryption

Hardcoded secrets

FAIL

password = "value" in .tf/.tfvars

08 Layer 5 — New Languages: Java · PHP · Ruby · Go · XML

Java .java

Runtime reflection & deserialization — critical attack surface

Runtime.exec() (CWE-78)

FAIL

OS command injection via Java runtime shell execution

ObjectInputStream (CWE-502)

FAIL

Deserialization of untrusted data — RCE vector

Log4Shell JNDI (CVE-2021-44228)

FAIL

JNDI lookup in log messages → critical 0-day pattern

TrustAllCerts (CWE-295)

FAIL

Disabled TLS certificate validation → MITM

PHP .php

Direct user input to shell / output — classic web attack surface

shell_exec() (CWE-78)

FAIL

Direct shell execution with user-controlled input

echo $_GET (CWE-79)

FAIL

Reflected XSS — user input echoed without sanitization

include($var) (CWE-22)

FAIL

Remote/local file inclusion via variable path

unserialize() (CWE-502)

FAIL

PHP deserialization → object injection & RCE

08 Layer 5 — New Languages: Java · PHP · Ruby · Go · XML

Ruby .rb

Metaprogramming power creates significant injection risk

system/exec (CWE-78) — FAIL

Shell command injection via system() or exec()

eval() (CWE-95) — FAIL

Dynamic code evaluation — trivial code injection

ActiveRecord interpolation (CWE-89) — FAIL

SQL injection via string interpolation in queries

Go .go

Low-level control — subtle SQL and crypto pitfalls

fmt.Sprintf in SQL (CWE-89) — FAIL

String-formatted SQL queries → SQL injection

math/rand (CWE-338) — WARN

Weak PRNG — not cryptographically secure

TLS MinVersion (CWE-326) — FAIL

Weak TLS version allowed (< TLS 1.2)

08 Layer 5 — New Languages: Java · PHP · Ruby · Go · XML

XML .xml

XML External Entity (XXE) — file read & SSRF via XML parsers

DOCTYPE + ENTITY (CWE-611)

FAIL

XXE: reads local files or makes internal HTTP requests

SYSTEM entity

FAIL

<!ENTITY xxe SYSTEM 'file:///etc/passwd'>

08 Layer 5 — Dockerfile · Binaries · Archives · SQL

Dockerfile

- **hadolint** — FAIL

CIS Docker: :latest, ADD http://, apt without --no-install-recommends

- **RUN curl|bash** — FAIL

Download + execution without integrity control

Binaries .so .exe .elf

- **Auto FAIL** — FAIL

No binaries in an AI repo — potential implant/RAT

- **strings + IOC** — FAIL

/bin/sh /etc/passwd exec reverse shell URLs detected

Archives .zip .tar.gz

- **Auto FAIL** — FAIL

Content cannot be scanned without prior extraction

- **Zip-slip** — FAIL

Extraction to arbitrary paths (../../etc/cron.d/)

SQL .sql

- **xp_cmdshell** — FAIL

Shell command execution from SQL Server → critical IOC

- **DROP TABLE / --** — FAIL

Destructive statements + SQL injection pattern

09 Tools Used — Summary

All optional in degraded mode — WARN if absent, FAIL only on active finding

Tool	Role	Installation	Scope
Gitleaks	secrets/tokens	GitHub binary	L1 Global
detect-secrets	high entropy	pip install	L1 Global
ClamAV	malware signatures	apt install clamav	L1 Global
YARA	custom IOC	apt install yara	L1 Global
Semgrep	OWASP/CWE rulesets (×4)	pip install semgrep	L2 OWASP/CWE
trivy	SCA universal CVE scan	GitHub binary	L3 SCA
pip-audit	Python SCA (PyPA)	pip install pip-audit	L3 SCA
safety	Python SCA (PyUp.io)	pip install safety	L3 SCA
npm audit	Node.js SCA (built-in)	built-in npm	L3 SCA
Bandit	Python SAST	pip install bandit	L5 .py
cppcheck	C/C++ static analysis	apt install cppcheck	L5 .c .cpp
ShellCheck	shell linting	apt install shellcheck	L5 .sh
hadolint	Dockerfile linting	GitHub binary	L5 Docker
checkov	IaC security	pip install checkov	L5 .tf
jq	GitHub API JSON parsing	apt install jq	Utility

10 Results — ZIP Archive & Excel Report

■ ZIP Archive → Good/

```
Good/
■■■ repo_20260615_143022.zip
    ■■■ src/
    ■   ■■■ [full source code]
    ■■■ README.md
    ■■■ scan_report_20260615.xlsx
```

- ✔ Produced only if verdict is PASS
- ✔ Timestamped name — guaranteed uniqueness
- ✔ .manifest_sha256.txt excluded
- ✔ Excel report included directly
- ✔ Ready for internal network transfer
- ✔ Optional: GPG encryption

■ Excel Report (.xlsx)

Tab 0 — Summary

Repository / Source	GitHub URL or local path
Scan Date	ISO 8601 timestamp
SHA-256 Hash	Global fingerprint of all files
Verdict	PASS (green) / FAIL (red)
Counters	PASS · WARN · FAIL

Tab 1 — Files (one row per file)

#	File	Type	Status	Message
1	src/main.py	.py	FAIL	bandit:HIGH
2	config.sh	.sh	WARN	shellcheck absent
3	README.md	.md	PASS	—

Sorted FAIL→WARN→PASS | Auto-filter | Row 1 frozen

11 Absolute Security Rules

■ **These rules CANNOT be modified — they constitute the threat model**

■ Network Isolation

The transit machine NEVER has direct access to the corporate network.

➡ One-way Flow

Only approved/ is readable from the inside — not fetch/, logs/, quarantine/.

■ Root-only Quarantine

chmod 700 — no application user can read rejected files.

■ No Manual Move

Never quarantine/ → approved/ without a full pipeline re-scan.

■ Binaries = Absolute FAIL

No .so/.exe/.elf in an AI repo — zero exceptions.

■ Archives = Mandatory Re-scan

.zip/.tar.gz → extraction + re-scan in a dedicated isolated environment.

Success Criteria per Tool

Success Criteria — Layer 1 (Global) & Layer 2 (OWASP/CWE)

Layer	Tool	PASS	WARN	FAIL	Standard
L1	gitleaks	0 secrets found	—	≥1 credential match	CWE-798 / OWASP A02
L1	detect-secrets	0 high-entropy hits	—	≥1 entropy hit	CWE-798 / OWASP A02
L1	ClamAV	0 signatures matched	DB not updated	≥1 malware signature	OWASP A08 / CERT
L1	YARA	0 IOC rules triggered	Custom rules absent	≥1 YARA rule hit	MITRE ATT&CK / IOC
L2	Semgrep p/owasp-top-ten	0 findings	Semgrep not found	≥1 HIGH/CRITICAL	OWASP Top 10 2021
L2	Semgrep p/cwe-top-25	0 findings	—	≥1 HIGH/CRITICAL	CWE Top 25
L2	Semgrep p/security-audit	0 findings	—	≥1 HIGH/CRITICAL	OWASP / CERT
L2	Semgrep p/secrets	0 findings	—	≥1 secret detected	CWE-798 / OWASP A02

All L1 findings → FAIL | L2 findings scored by severity: HIGH/CRITICAL → FAIL, MEDIUM → WARN

Success Criteria per Tool

Success Criteria — Layer 3 (SCA) & Layer 4 (Universal Patterns)

Layer	Tool	PASS	WARN	FAIL	CWE / OWASP
L3	trivy	0 CVEs CRITICAL/HIGH	MEDIUM CVE found	≥1 CRITICAL/HIGH CVE	OWASP A06 / NVD
L3	pip-audit / safety	0 vulnerable pkgs	Outdated package	≥1 known CVE in reqs	OWASP A06 / PyPA
L3	npm audit	0 critical/high	moderate advisory	critical or high vuln	OWASP A06 / npm
L4	Hardcoded creds	0 matches	—	password= / api_key=	CWE-798
L4	Hardcoded crypto	0 matches	—	BEGIN RSA PRIVATE KEY	CWE-321
L4	Path traversal	0 matches	—	../../etc/passwd	CWE-22
L4	SSRF pattern	0 matches	user_input in URL	—	CWE-918 / OWASP A10
L4	Weak crypto	0 matches	md5 / sha1 / rc4	—	CWE-327
L4	Weak PRNG	0 matches	Math.random()/rand()	—	CWE-338
L4	Logging disabled	0 matches	logging.disable(…)	—	OWASP A09

L3: CRITICAL/HIGH CVE → FAIL | L4: hardcoded secrets/traversal → FAIL; weak patterns → WARN

Success Criteria per Tool

Success Criteria — Layer 5 (Per-type SAST) — Summary Matrix

Lang / Type	Key Tool	# Checks	Verdict if triggered	Key Standards
L5 **Python .py**	Bandit	50+	FAIL	CWE-78, CWE-89, CWE-94, OWASP A03
L5 **JS / TS .js .ts**	Semgrep	40+	FAIL	CWE-79, CWE-94, OWASP A03 XSS/Injection
L5 **C / C++ .c .cpp**	cppcheck	30+	FAIL	CWE-120, CWE-125, CWE-416, CERT-C
L5 **Shell .sh .bash**	ShellCheck	20+	FAIL	CWE-78, OWASP A03, CERT
L5 **Java .java**	Semgrep	25+	FAIL	CWE-78, CWE-502, CVE-2021-44228, CWE-295
L5 **PHP .php**	Semgrep	20+	FAIL	CWE-78, CWE-79, CWE-22, CWE-502
L5 **Ruby .rb**	Semgrep	15+	FAIL	CWE-78, CWE-89, CWE-95
L5 **Go .go**	Semgrep	15+	FAIL/WARN	CWE-89, CWE-326, CWE-338
L5 **XML .xml**	grep (regex)	2	FAIL	CWE-611 (XXE)
L5 **YAML .yml**	— (regex)	2	FAIL	Supply-chain / OWASP A08
L5 **Terraform .tf**	checkov	50+	FAIL	CIS Benchmarks, OWASP A05, CWE-798
L5 **Dockerfile**	hadolint	30+	FAIL	CIS Docker Benchmark, OWASP A05
L5 **SQL .sql**	grep (regex)	3	FAIL	CWE-89, CWE-78
L5 **Binary .so .exe**	strings+Auto	—	FAIL ■	Auto-FAIL policy — no binaries in AI repos
L5 **Archive .zip .tar**	— (policy)	—	FAIL ■	Auto-FAIL — re-scan after extraction

All L5 positive findings → FAIL | Missing tool → WARN (degraded mode) | Binaries & Archives → Auto-FAIL

betterleaks

PURPOSE

Detects secrets, credentials and API keys embedded in source files. Successor to gitleaks — maintained by the same author with a more expressive allowlist system. Scans all file types, not just git history.

INSTALL

```
go install github.com/betterleaks/betterleaks@latest
  brew install betterleaks
```

SCAN COMMAND

```
betterleaks dir <repo_dir> -v
```

✔ Works OFFLINE

✓ PASS condition

Exit code 0 — no secrets or credentials detected in any file.

✘ FAIL condition → pipeline blocked

Exit code ≠ 0 — at least one secret pattern matched (API key, token, password, private key …). → Pipeline verdict: FAIL.

■ WARN condition → logged, pipeline continues

Tool not installed → scan skipped, WARN logged (degraded mode).

detect-secrets

PURPOSE

Entropy-based secret detector — complements betterleaks with a different detection engine. Uses Shannon entropy and regex rules to find high-entropy strings that look like secrets.

INSTALL

```
pip install detect-secrets
```

SCAN COMMAND

```
detect-secrets scan <repo_dir>
```

✔ Works OFFLINE

✓ PASS condition

JSON output with empty 'results' dict — no high-entropy secrets found.

✘ FAIL condition → pipeline blocked

JSON output lists one or more findings → Pipeline verdict: FAIL.

■ WARN condition → logged, pipeline continues

Tool not installed → scan skipped, WARN logged.

ClamAV (clamscan)

PURPOSE

Open-source antivirus engine. Scans all files against a database of known malware signatures. Requires periodic DB updates via freshclam to stay effective against recent threats.

INSTALL

```
sudo apt-get install clamav clamav-daemon
sudo freshclam   # update DB
```

SCAN COMMAND

```
clamscan -r --no-summary <repo_dir>
```

✔ Works OFFLINE

✓ PASS condition

Exit code 0 — no virus signatures matched.

✘ FAIL condition → pipeline blocked

Exit code 1 — at least one infected file found → Pipeline verdict: FAIL.

■ WARN condition → logged, pipeline continues

freshclam DB not updated (>7 days) → WARN. Tool missing → WARN.

YARA

PURPOSE

Pattern-matching engine for malware and IOC detection. Loads all .yar rule files from yara-rules/ and matches them against every file. Rules are fully customisable — add organisation-specific IOC patterns.

INSTALL

```
sudo apt-get install yara
```

SCAN COMMAND

```
yara -r <rules_dir>/*.yar <repo_dir>
```

✔ Works OFFLINE

✓ PASS condition

No output — no rule matched any file.

✘ FAIL condition → pipeline blocked

At least one YARA rule matched → Pipeline verdict: FAIL.

■ WARN condition → logged, pipeline continues

No .yar files in yara-rules/ → WARN. Tool missing → WARN.

Semgrep

PURPOSE

Static analysis engine with a large open rule library.

Runs four rulesets: p/owasp-top-ten, p/cwe-top-25, p/security-audit, p/secrets. Covers injection, XSS, insecure deserialization, SSRF, weak crypto and 100+ more patterns.

INSTALL

```
pip install semgrep
```

SCAN COMMAND

```
semgrep --config p/owasp-top-ten --config p/cwe-top-25
  --config p/security-audit --config p/secrets
  --json <repo_dir>
```

■ Requires INTERNET (first run)

✓ PASS condition

JSON output with empty 'results' list — no rule matched.

✘ FAIL condition → pipeline blocked

ERROR or WARNING severity findings → Pipeline verdict: FAIL.

■ WARN condition → logged, pipeline continues

INFO severity → WARN. Tool missing or no internet → WARN.

trivy

PURPOSE

Software Composition Analysis — scans dependency manifests (requirements.txt, package-lock.json, go.sum, pom.xml …) and cross-references installed packages against NVD, OSV and GitHub Advisory databases to find known CVEs.

INSTALL

```
curl -sfL https://raw.githubusercontent.com/aquasecurity/trivy/
  main/contrib/install.sh | sh -s -- -b /usr/local/bin
```

SCAN COMMAND

```
trivy fs <repo_dir> --scanners vuln
  --severity HIGH,CRITICAL --format json
```

■ Requires INTERNET (first run)

✓ PASS condition

JSON output with no vulnerabilities at HIGH or CRITICAL severity.

✘ FAIL condition → pipeline blocked

CRITICAL or HIGH CVE found in a dependency → Pipeline verdict: FAIL.

■ WARN condition → logged, pipeline continues

MEDIUM/LOW CVE → WARN. DB not up to date → WARN. Missing → WARN.

pip-audit

PURPOSE

Audits Python requirements files against the Python Packaging Advisory Database (PyPA) and OSV. Checks requirements.txt, requirements*.txt and setup.cfg for packages with known CVEs.

INSTALL

```
pip install pip-audit
```

SCAN COMMAND

```
pip-audit -r requirements.txt --format json
```

■ Requires INTERNET (first run)

✓ PASS condition

JSON output with no vulnerabilities listed.

✘ FAIL condition → pipeline blocked

Any CVE found in a Python dependency → Pipeline verdict: FAIL.

■ WARN condition → logged, pipeline continues

Tool missing → WARN, scan skipped.

safety

PURPOSE

Checks Python dependencies against the Safety DB — a curated advisory database maintained by PyUp.io. Complements pip-audit with additional advisories not always in OSV.

INSTALL

```
pip install safety
```

SCAN COMMAND

```
safety check -r requirements.txt --json
```

■ Requires INTERNET (first run)

✓ PASS condition

Exit code 0 — no advisories matched.

✘ FAIL condition → pipeline blocked

Any advisory matched → Pipeline verdict: FAIL.

■ WARN condition → logged, pipeline continues

Tool missing or DB unreachable → WARN.

npm audit

PURPOSE

Built into npm — audits package-lock.json against the npm security advisory registry. Finds known CVEs in Node.js dependencies and their transitive dependencies.

INSTALL

```
sudo apt-get install nodejs npm
(or: nvm install --lts)
```

SCAN COMMAND

```
npm audit --json   (run in dir with package-lock.json)
```

■ Requires INTERNET (first run)

✓ PASS condition

JSON output with 0 vulnerabilities at high or critical level.

✘ FAIL condition → pipeline blocked

Critical or high severity advisory found → Pipeline verdict: FAIL.

■ WARN condition → logged, pipeline continues

Moderate/low → WARN. No package-lock.json → scan skipped.

grep (universal patterns)

PURPOSE

Runs on every file regardless of extension. Checks for:

CWE-798 (hardcoded credentials), CWE-22 (path traversal),

CWE-918 (SSRF patterns), CWE-327 (weak crypto constants),

CWE-338 (weak PRNG), OWASP-A09 (logging of sensitive data).

INSTALL

```
Built-in — no installation required
```

SCAN COMMAND

```
grep -rE '<pattern>' <file>   (run per-pattern, per-file)
```

✔ Works OFFLINE

✓ PASS condition

No pattern matches across all CWE checks for this file.

✘ FAIL condition → pipeline blocked

Any pattern match → finding recorded. Severity determines WARN or FAIL.

■ WARN condition → logged, pipeline continues

N/A — grep is always available.

Bandit

PURPOSE

Python-specific static analyser. Detects common security issues:
use of assert, subprocess with shell=True, hardcoded passwords,
use of pickle, XML vulnerabilities, SQL injection via string format,
insecure random, weak hashing, and more (100+ plugins).

INSTALL

```
pip install bandit
```

SCAN COMMAND

```
bandit -ll -r <python_file_or_dir>
```

✔ Works OFFLINE

✔ PASS condition

Exit code 0 — no medium or high confidence issues found.

✘ FAIL condition → pipeline blocked

MEDIUM or HIGH severity + MEDIUM or HIGH confidence → FAIL.

■ WARN condition → logged, pipeline continues

LOW confidence findings → WARN. Tool missing → WARN.

ShellCheck

PURPOSE

Static analyser for Bash/sh/dash/ksh scripts. Detects quoting errors, injection risks, undefined variables, deprecated syntax, unintended word splitting, and unsafe patterns that could lead to command injection or data leakage.

INSTALL

```
sudo apt-get install shellcheck
```

SCAN COMMAND

```
shellcheck --severity=warning <script.sh>
```

✔ Works OFFLINE

✓ PASS condition

Exit code 0 — no warnings or errors.

✘ FAIL condition → pipeline blocked

Exit code ≠ 0 (errors/warnings at warning level or above) → FAIL.

■ WARN condition → logged, pipeline continues

Tool missing → WARN.

cppcheck

PURPOSE

Static analyser for C and C++. Detects buffer overflows, memory leaks, null-pointer dereferences, use-after-free, out-of-bounds array access, and undefined behaviour — without compiling the code.

INSTALL

```
sudo apt-get install cppcheck
```

SCAN COMMAND

```
cppcheck --enable=warning,security --error-exitcode=1 <file.cpp>
```

✔ Works OFFLINE

✓ PASS condition

Exit code 0 — no errors or security warnings.

✘ FAIL condition → pipeline blocked

Exit code 1 — at least one security or error finding → FAIL.

■ WARN condition → logged, pipeline continues

Tool missing → WARN.

hadolint

PURPOSE

Linter for Dockerfiles. Enforces Dockerfile best practices and detects security misconfigurations: running as root, using :latest tags, unnecessary privileges, secrets in ENV/ARG, ADD vs COPY, and shell injection risks.

INSTALL

```
curl -sSL https://github.com/hadolint/hadolint/releases/
  download/v2.12.0/hadolint-Linux-x86_64 -o /usr/local/bin/hadolint
```

SCAN COMMAND

```
hadolint --failure-threshold warning <Dockerfile>
```

✔ Works OFFLINE

✓ PASS condition

Exit code 0 — no warnings or errors.

✘ FAIL condition → pipeline blocked

DL or SC rules triggered at warning level or above → FAIL.

■ WARN condition → logged, pipeline continues

Tool missing → WARN.

checkov

PURPOSE

Static analysis for Infrastructure-as-Code: Terraform (.tf), CloudFormation, Kubernetes YAML, Ansible, ARM templates. Maps findings to CIS Benchmarks, NIST, SOC2 and OWASP. Over 1000 built-in checks.

INSTALL

```
pip install checkov
```

SCAN COMMAND

```
checkov -d <dir> --framework terraform --output json
```

✔ Works OFFLINE

✔ PASS condition

All checks pass — no FAILED checks in JSON output.

✘ FAIL condition → pipeline blocked

At least one FAILED check → Pipeline verdict: FAIL.

■ WARN condition → logged, pipeline continues

SKIPPED checks → WARN. Tool missing → WARN.

ScanCode Toolkit

PURPOSE

Full-text licence and copyright analyser. Detects SPDX licence expressions across all source files using a library of 30 000+ licence texts. Also detects package manifests and CVEs in detected packages. Produces JSON / SPDX / CycloneDX reports.

INSTALL

```
pip install scancode-toolkit
```

SCAN COMMAND

```
scancode --license --copyright --vulnerability
  --package --json-pp report.json <repo_dir>
```

✔ Works OFFLINE

✓ PASS condition

No risky licences (GPL/AGPL/LGPL/SSPL …) detected AND no HIGH/CRITICAL CVE in detected packages.

✘ FAIL condition → pipeline blocked

CRITICAL or HIGH CVE in a detected package → FAIL.

■ WARN condition → logged, pipeline continues

Risky licence detected (score ≥ 70) → WARN (legal review required).

LOW/MEDIUM CVE → WARN. Tool missing → WARN.

Tool Summary — Acceptance Rationale

Tool	Layer	Offline	✔ PASS	✘ FAIL	■ WARN
betterleaks	L1	✔	Exit 0 — no secrets	Secret/key detected → FAIL	Not installed → WARN
detect-secrets	L1	✔	Empty results JSON	High-entropy string found → FAIL	Not installed → WARN
ClamAV	L1	✔	Exit 0 — no virus	Infected file found → FAIL	DB outdated / missing → WARN
YARA	L1	✔	No rule matched	IOC rule matched → FAIL	No rules / missing → WARN
Semgrep	L2	■	Empty results list	ERROR/WARNING severity → FAIL	INFO / no internet → WARN
trivy	L3	■	No HIGH/CRITICAL CVE	CRITICAL or HIGH CVE → FAIL	MEDIUM/LOW CVE → WARN
pip-audit	L3	■	No CVE in Python deps	Any Python CVE → FAIL	Not installed → WARN
safety	L3	■	No advisory matched	Advisory matched → FAIL	Not installed → WARN
npm audit	L3	■	0 critical/high vulns	Critical/high vuln → FAIL	No package-lock.json → skip
grep (patterns)	L4	✔	No CWE pattern matched	CWE-798/22/918/327/338 hit → FAIL	N/A — always available
Bandit	L5	✔	No medium/high issues	Medium+ severity + confidence → FAIL	Low confidence → WARN
ShellCheck	L5	✔	Exit 0 — no warnings	Warning-level finding → FAIL	Not installed → WARN
cppcheck	L5	✔	No errors/security warns	Security/error finding → FAIL	Not installed → WARN
hadolint	L5	✔	Exit 0 — no warnings	DL/SC rule triggered → FAIL	Not installed → WARN
checkov	L5	✔	All IaC checks pass	Any FAILED check → FAIL	SKIPPED checks → WARN
PDF scanner	L5	✔	No active content	JavaScript / auto-action / Launch / embedded exe → FAIL	Encrypted or undecodable stream → WARN
ScanCode	L6	✔	No risky licence, no CVE	CRITICAL/HIGH CVE in pkg → FAIL	Risky licence (GPL…) → WARN

FAIL = package blocked, moved to quarantine | WARN = logged, pipeline continues (degraded mode) | All layers must PASS for the ZIP to be released

Operating the Pipeline

The same scan, tuned to the situation it runs in

FLAG	EFFECT	TYPICAL USE
`--quiet`	Verdict only — nothing else on stdout	CI gate
`--verbose`	Full per-file detail	Debugging a finding
`--min-severity LEVEL`	low | medium | high | critical	Tune the blocking bar
`--since COMMIT`	Scan only files changed since COMMIT	Pull-request checks
`--report-only`	Always exit 0; nothing quarantined	First-pass audit
`--no-zip / --no-excel`	Reports only, no archive	CI, where the ZIP is discarded

Per-repository controls

.transitignore gitignore-style patterns — matched files are not scanned at all

.transit-allow.json {rule, path, reason} entries — downgrade a known FAIL to WARN

An allowlist entry records why a finding is accepted. Silence without a reason is how exceptions become permanent.

How the Pipeline Verifies Itself

A scanner nobody tests is a scanner nobody should trust

Test suite

84 assertions

- Runs with zero scanning tools installed
- Rule corpus: every finding must land on the right file
- False-positive guards for correct, safe code
- Flags, reports, archive structure, diff mode

Continuous integration

5 jobs

- lint — shellcheck, errors fatal
- test — suite, no tools (fast signal)
- test-with-tools — scanners installed
- pins — pinned versions resolve + digest match
- docker — build, non-root, smoke tests

Every assertion is validated by mutation

The bug each test guards against is deliberately reintroduced, and the test is confirmed to fail.
Two tests were themselves broken when first written — they passed against known-broken code.
A suite that has never been seen red proves nothing.

Lint rules enforce two bug classes that already shipped:

multi-character IFS — IFS=':::' is a character set, silently identical to IFS=':'
top-level 'local' — valid syntax, but aborts at runtime under set -e

Air-Gapped Operation

Disconnected scanning, and the failure it is designed to prevent

The risk

Several scanners fetch at SCAN time: semgrep pulls rulesets, trivy downloads a CVE database, pip-audit / safety / npm audit query advisory services. On an isolated host those calls do not fail loudly — they block on timeouts, return nothing, and Layers 2 and 3 contribute no findings while the report still shows them as having run. The result is PASS on a repository never examined.

No preparation

- betterleaks, detect-secrets
- YARA, pattern rules
- Bandit, ShellCheck
- cppcheck, hadolint

Rules are local

Needs staged data

- Semgrep (ruleset YAML)
- trivy (CVE database)
- ClamAV (signatures)
- checkov, ScanCode

Staged by prepare_offline_cache.sh

No offline mode

- pip-audit
- safety
- npm audit
- ScanCode CVE lookup

Not attempted; gap is recorded

Offline, the staged trivy database is the ONLY dependency-CVE coverage there is.

Proving the scan was real

Every report carries a coverage block: "L2_owasp_cwe": "ran" · "L3_dependency_cve": "skipped:trivy database not staged"

A layer can fail to run because the tool is absent OR its data was not staged — both are recorded identically, so automated consumers gate on coverage_complete rather than parsing warning text. A verdict alone cannot distinguish “clean” from “nothing was examined”.

Summary

Source

GitHub (whitelist)

or local path

Fetch

Minimal clone

.git removed · SHA-256

6 Layers

L1→L2→L3→L4→L5→L6

16 tools · SCA · Licence

PASS

ZIP in Good/

with Excel included

FAIL

Quarantine

chmod 700 + report

OWASP | CWE | CERT | SCA

6-layer pipeline · OWASP · CWE · CERT · SCA · Licence (ScanCode) · Degraded mode · Timestamped reports

github.com/Gaillotte/Claude — branch claude/vigilant-carson-f8twy0